

AI WEATHERWISE

Phase 1: Brainstorming & Ideation

Problem discovery, idea generation and solution selection

Project	AI WeatherWise – Deliver intelligent weather forecasts and insights
Phase	1 of 8 – Brainstorming & Ideation
Technology	Node.js, Express.js, MongoDB (Mongoose), JWT, Google Gemini AI, OpenWeatherMap
Version / Date	1.0 28 September 2026

1. Purpose of This Phase

The Brainstorming & Ideation phase captures how the AI WeatherWise idea was born, which problems it solves, who it is built for, and which ideas were selected for the first release. It is the foundation for the requirement, design and development phases that follow.

2. Background and Problem Statement

Alex is a travel enthusiast who frequently visits different cities. Alex needs a streamlined way to track the local weather of many places and to receive activity suggestions that match the current forecast. Existing weather tools return raw numbers and leave the interpretation to the user.

#	Problem	Impact on the user
P1	Difficulty tracking and managing multiple favorite weather locations efficiently	Users re-search the same cities again and again; no single place to manage them.
P2	Lack of personalized recommendations for outdoor activities or clothing	Users must decide by themselves what to wear or plan, based only on numbers.
P3	Need for natural language weather summaries instead of raw data metrics	Temperature, humidity and wind values take time and effort to interpret.
P4	Absence of a secure system to store and sync preferences across devices	Saved cities and preferences are lost when the user changes device.

3. Target Users and Personas

Persona	Goals	Pain points / needs
Alex – Travel enthusiast (Registered User)	Track weather of several cities, get quick advice on clothing and outdoor plans, access the same list on any device.	Too many raw numbers, no personalization, no secure sync.
Weather Application User (Public)	Read current weather metrics quickly without creating an account.	Does not want to register just to see the weather.

Persona	Goals	Pain points / needs
Developer / Admin	Monitor API health, manage user registrations and verify weather and AI integrations.	Needs visibility, secure access control and reliable integrations.

3.1 Empathy Map (Alex)

Dimension	Observation
Says	"What should I wear in Bangalore today?" "Which of my cities has the best weather this weekend?"
Thinks	Numbers alone do not tell me what to do. I wish the app simply advised me.
Does	Checks several weather sites, keeps notes of cities, switches between devices while travelling.
Feels	Frustrated by scattered information, and worried about planning outdoor activities badly.

4. Brainstorming Session

The team brainstormed features around three questions: "How can we save time for the traveller?", "How can raw data become advice?" and "How can we keep user data safe?" The ideas generated are listed below.

Theme	Ideas generated
Location management	Save favorite cities; edit or delete a saved city; prevent duplicate cities per user; group cities by country.
Weather data	Real-time weather from OpenWeatherMap; temperature, humidity, wind speed and condition; public endpoint that needs no login.
AI intelligence	Natural-language weather summary; clothing advice; outdoor activity suggestions using Google Gemini.
Security	JWT authentication; bcrypt password hashing; role-based route guarding; request sanitization against XSS and NoSQL injection.
Reliability	Fallback (mock) mode so the API still works without external API keys; centralized error handling.
Administration	User account management; API health and usage monitoring; database integrity checks.
Future ideas	Weather alerts, subscription plans, weather history, AI model management, API usage logs, mobile or web front end.

5. Idea Prioritization

Ideas were grouped using the MoSCoW method so that the first release stays small and deliverable.

Priority	Ideas	Reason
Must have	User registration and login (JWT); favorite locations CRUD; real-time weather; AI recommendation; fallback mode	Core value of the product and required for the end-to-end user flow.

Priority	Ideas	Reason
Should have	AI weather summary; profile update; centralized error handling; input sanitization	Improves experience and protects the system.
Could have	Admin monitoring of API health; role matrix for admin actions	Useful for operations but not needed for the first demo.
Won't have (now)	Alerts, subscriptions, weather history, API usage logs, AI model registry, front-end UI	Planned for future scope (see the extended ER diagram).

6. Proposed Solution

AI WeatherWise is a robust RESTful backend platform built with Node.js and Express.js, using MongoDB with the Mongoose ODM for data persistence. It retrieves real-time weather metrics and combines them with Google Gemini AI to provide intelligent summaries and personalized recommendations. Users store favorite cities securely and can fetch weather insights from any device through the API.

6.1 Value Proposition

- **Centralized:** one service for favorite locations, weather retrieval and AI insights.
- **Intelligent:** turns raw metrics into plain-language advice about clothing and activities.
- **Secure:** JWT authentication and bcrypt hashing protect accounts and saved data.
- **Resilient:** fallback mode keeps the API working even without external API keys.

6.2 Scenario Walkthrough

Alex registers, saves a favorite city, fetches its current weather and asks the AI for a recommendation. The system validates the JWT and returns the information, so Alex spends less time deciphering numbers.

AI WeatherWise API User Workflow

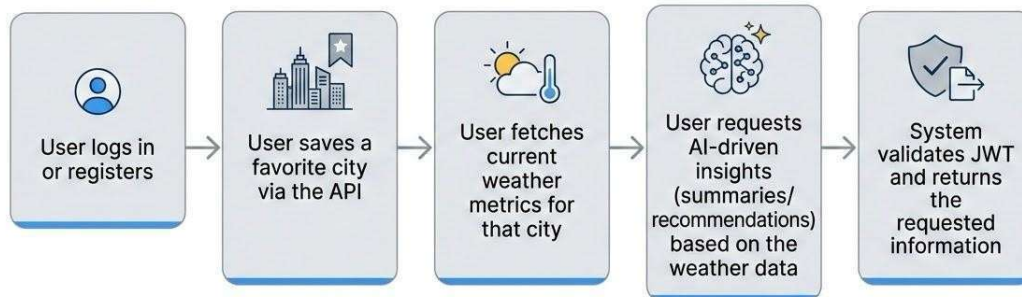


Figure 1: AI WeatherWise API user workflow

7. Technology Ideation

Need	Options considered	Decision
Server runtime	Node.js / Python Flask / Java Spring	Node.js + Express.js – fast, JavaScript everywhere, large ecosystem.
Database	MongoDB / MySQL	MongoDB + Mongoose – flexible documents, simple user–location references.
Authentication	Sessions / JWT	JWT + bcryptjs – stateless, works across devices.
Weather source	OpenWeatherMap / other APIs	OpenWeatherMap – current weather metrics, simple REST API.
AI engine	Google Gemini / other LLMs	Google Gemini – natural-language summaries and recommendations.
API testing	Postman / Thunder Client	Postman – request collections and response verification.

8. Expected Outcomes and Success Criteria

- A user can register, log in and receive a JWT token.
- A user can save, view, update and delete favorite cities without duplicates.
- Current weather is returned for a city within a second.
- The AI service returns a clear recommendation for the given weather.
- The API keeps working in fallback mode when no external key is configured.

9. Conclusion and Next Phase

The ideation phase confirmed a clear problem, defined the target users and selected a focused first-release scope. The next phase, **Requirement Analysis**, converts these ideas into detailed functional and non-functional requirements.